

WEEK 1 – ENVIRONMENT SETUP AND RECONNAISSANCE

Web Application Penetration Testing is an important cybersecurity practice used to identify vulnerabilities present in web applications before attackers can exploit them. Modern web applications often contain security weaknesses related to authentication, authorization, input validation, and data protection.

To understand these vulnerabilities in a practical manner, a testing environment was created using OWASP Juice Shop and DVWA (Damn Vulnerable Web Application). Burp Suite was used for web application traffic analysis and reconnaissance, while Nmap was used to identify active services running on the target environment.

This week mainly focused on understanding the OWASP Top 10 vulnerabilities, configuring the penetration testing environment, and performing reconnaissance activities required before vulnerability testing.

The primary objectives of Week 1 were:

- To study OWASP Top 10 Web Application Security Risks.
- To set up a vulnerable web application testing environment.
- To deploy OWASP Juice Shop using Docker.
- To verify Apache and MariaDB services.
- To configure Burp Suite as an intercepting proxy.
- To perform reconnaissance and traffic analysis.
- To identify application endpoints for future testing.

The Tools Used Are

Tool	Purpose
Kali Linux	Penetration Testing Platform
OWASP Juice Shop	Vulnerable Web Application
DVWA	Vulnerable Web Application
Docker	Container Deployment
Burp Suite Community Edition	Web Traffic Interception and Analysis
Firefox Browser	Accessing Web Applications
Nmap	Service Discovery and Reconnaissance
Apache2	Web Server
MariaDB	Database Server

Study of OWASP Top 10 Vulnerabilities

Before starting practical activities, the OWASP Top 10 security risks were studied to understand the most common vulnerabilities affecting modern web applications.

The studied vulnerabilities included:

- Broken Access Control
- Cryptographic Failures
- Injection Attacks
- Insecure Design
- Security Misconfiguration
- Vulnerable Components
- Identification and Authentication Failures
- Software and Data Integrity Failures
- Security Logging and Monitoring Failures
- Server-Side Request Forgery (SSRF)

This theoretical understanding provided the foundation required for practical penetration testing activities.

OWASP Juice Shop Deployment

OWASP Juice Shop was selected because it is one of the most widely used intentionally vulnerable web applications designed for cybersecurity training and penetration testing practice.

Commands Used

- Check Docker Installation

```
"docker -version"
```

- Download OWASP Juice Shop Image

```
"docker pull bkimminich/juice-shop"
```

- Run Juice Shop Container

```
"docker run -d -p 3000:3000 bkimminich/juice-shop"
```

- Verify Running Containers

```
"docker ps"
```

- Access Application

```
"http://127.0.0.1:3000"
```

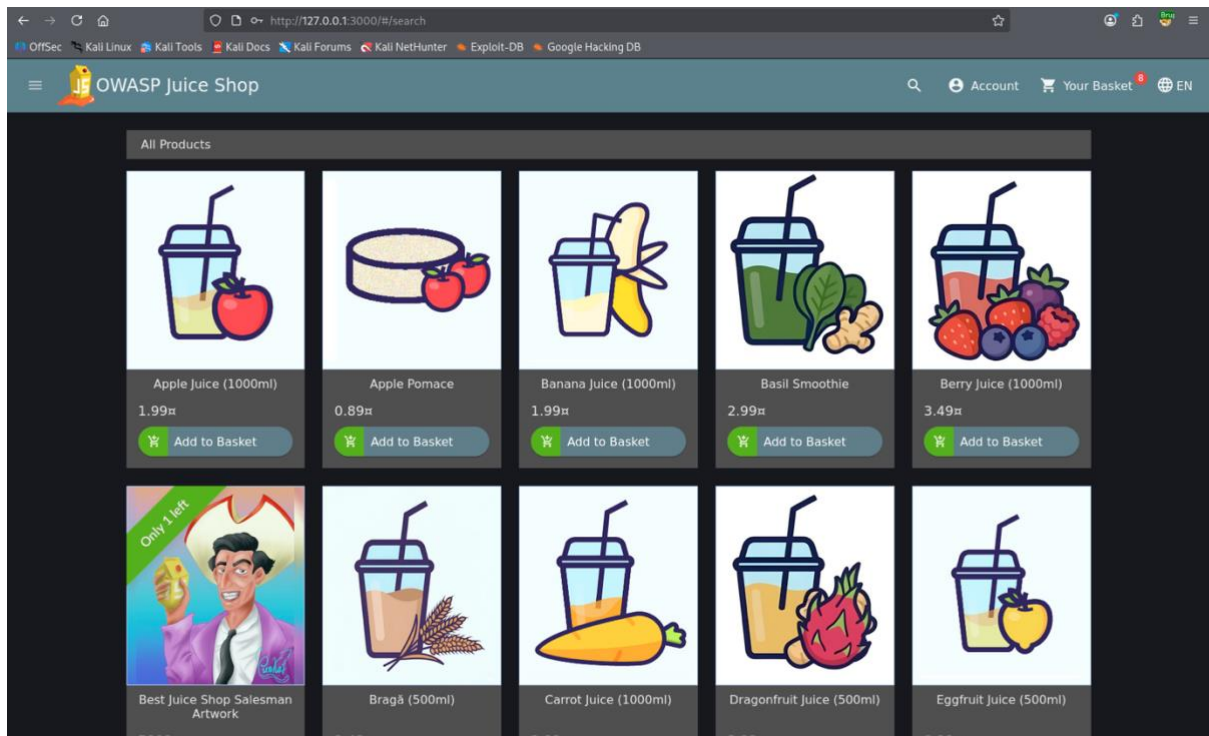


Figure 1: OWASP Juice Shop Homepage

```
(root@kali)-[~]
# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c192fa9acbb9	bkimminich/juice-shop	"/nodejs/bin/node /j..."	16 hours ago	Up 16 hours	0.0.0.0:3000→3000/tcp, [::]:3000→3000/tcp
fervent_snyder					

Figure 2: Docker Container Running Successfully

Service Verification

After configuring the environment, essential services were verified.

Commands Used

- Check Apache Service

```
"systemctl status apache2"
```

- Check MariaDB Service

```
"systemctl status mariadb"
```

- Check Active Listening Ports

"ss -tulnp"

Findings

Port	Service
80	Apache HTTP Server
3000	OWASP Juice Shop
3306	MariaDB Database Server

```
(root@kali)-[/var/www/html]
# sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-05-30 18:30:12 IST; 1min 11s ago
 Invocation: c1643c0192f843b8a5932c3b55abc3b5
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 119699 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 6 (limit: 7073)
  Memory: 20.7M (peak: 20.9M)
     CPU: 59ms
   CGroup: /system.slice/apache2.service
           └─119699 /usr/sbin/apache2 -k start -DFOREGROUND
             └─119702 /usr/sbin/apache2 -k start -DFOREGROUND
               └─119703 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─119704 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─119705 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─119706 /usr/sbin/apache2 -k start -DFOREGROUND

May 30 18:30:12 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 30 18:30:12 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Figure 3: Apache Service Running

```
(root@kali)-[/var/www/html]
# sudo systemctl start apache2

(root@kali)-[/var/www/html]
# sudo systemctl start mariadb

(root@kali)-[/var/www/html]
# sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-05-30 18:30:25 IST; 11s ago
     Invocation: 77634031ec444ff8bd4f423ea0a2313e
       Docs: man:mariadb(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 119826 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR="/usr/bin/galera_recovery"; [ $
    Process: 119896 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /run/mysqld/wsrep-new-cluster (code=exited, stat
    Process: 119898 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
   Main PID: 119882 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 14 (limit: 46682)
    Memory: 138.1M (peak: 143.3M)
       CPU: 860ms
    CGroup: /system.slice/mariadb.service
            └─119882 /usr/sbin/mariabdd

May 30 18:30:24 kali mariabdd[119882]: 2026-05-30 18:30:24 0 [Note] InnoDB: log sequence number 45141; transaction id 14
May 30 18:30:24 kali mariabdd[119882]: 2026-05-30 18:30:24 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buff
May 30 18:30:24 kali mariabdd[119882]: 2026-05-30 18:30:24 0 [Note] Plugin 'FEEDBACK' is disabled.
May 30 18:30:24 kali mariabdd[119882]: 2026-05-30 18:30:24 0 [Note] Plugin 'wsrep-provider' is disabled.
May 30 18:30:24 kali mariabdd[119882]: 2026-05-30 18:30:24 0 [Note] InnoDB: Buffer pool(s) load completed at 260530 18:30:24
May 30 18:30:25 kali mariabdd[119882]: 2026-05-30 18:30:25 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
May 30 18:30:25 kali mariabdd[119882]: 2026-05-30 18:30:25 0 [Note] mariabdd: Event Scheduler: Loaded 0 events
May 30 18:30:25 kali mariabdd[119882]: 2026-05-30 18:30:25 0 [Note] /usr/sbin/mariabdd: ready for connections.
May 30 18:30:25 kali mariabdd[119882]: Version: '11.8.6-MariaDB-6 from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306
May 30 18:30:25 kali systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
lines 1-27/27 (END)
```

Figure 4: MariaDB Service Running

Reconnaissance Using Nmap

Reconnaissance is the first phase of penetration testing where information about the target environment is collected.

Command Used

```
"nmap -sV 127.0.0.1"
```

Results

The Nmap scan successfully identified:

- Apache HTTP Server on Port 80
- OWASP Juice Shop on Port 3000
- MariaDB Database Service on Port 3306

```
(root@kali)-[~]
# nmap localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-30 18:39 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000020s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
3306/tcp  open  mysql

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

Figure 5: Nmap Service Version Detection Scan

Burp Suite Configuration

Burp Suite Community Edition was configured as an intercepting proxy to analyze web application traffic.

Launch Burp Suite

```
"burpsuite"
```

Browser Proxy Settings

```
"HTTP Proxy : 127.0.0.1"
"Port : 8080"
```

Activities Performed

- Configured Firefox Proxy Settings
- Enabled Request Interception
- Captured HTTP Requests
- Monitored HTTP Responses
- Analyzed API Endpoints

Burp Suite allowed complete visibility into communication between the browser and the web application. It helped identify API endpoints, request parameters, and application behavior which will be useful during vulnerability testing.

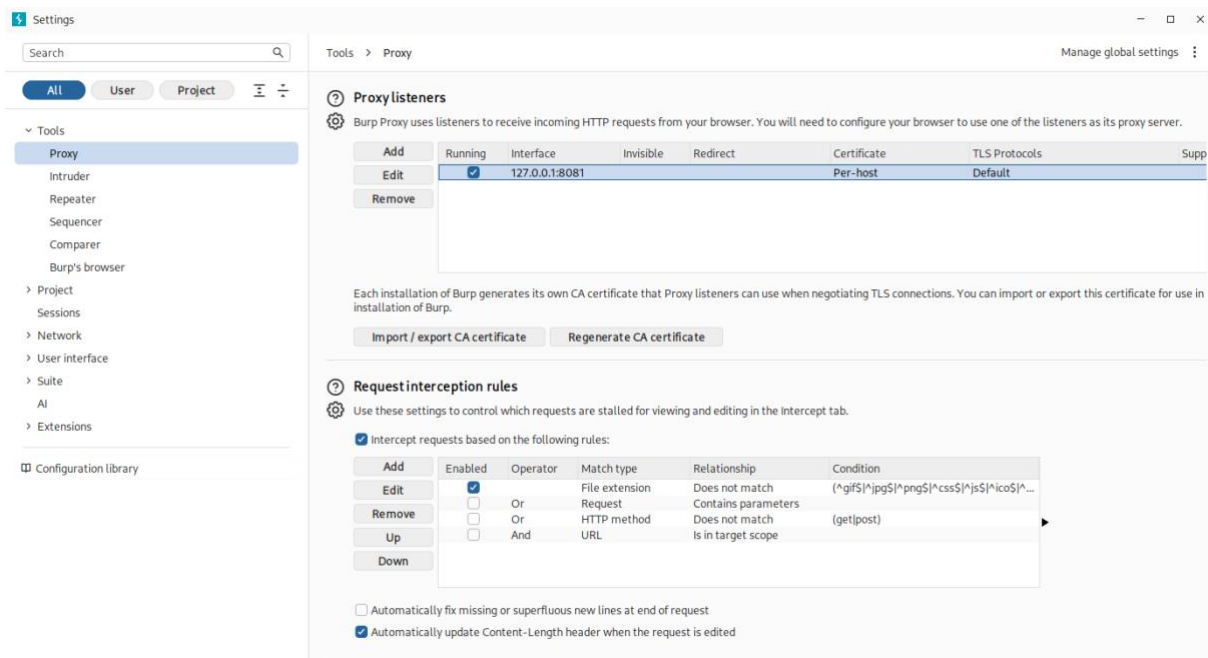


Figure 6: Burp Suite Proxy Configuration

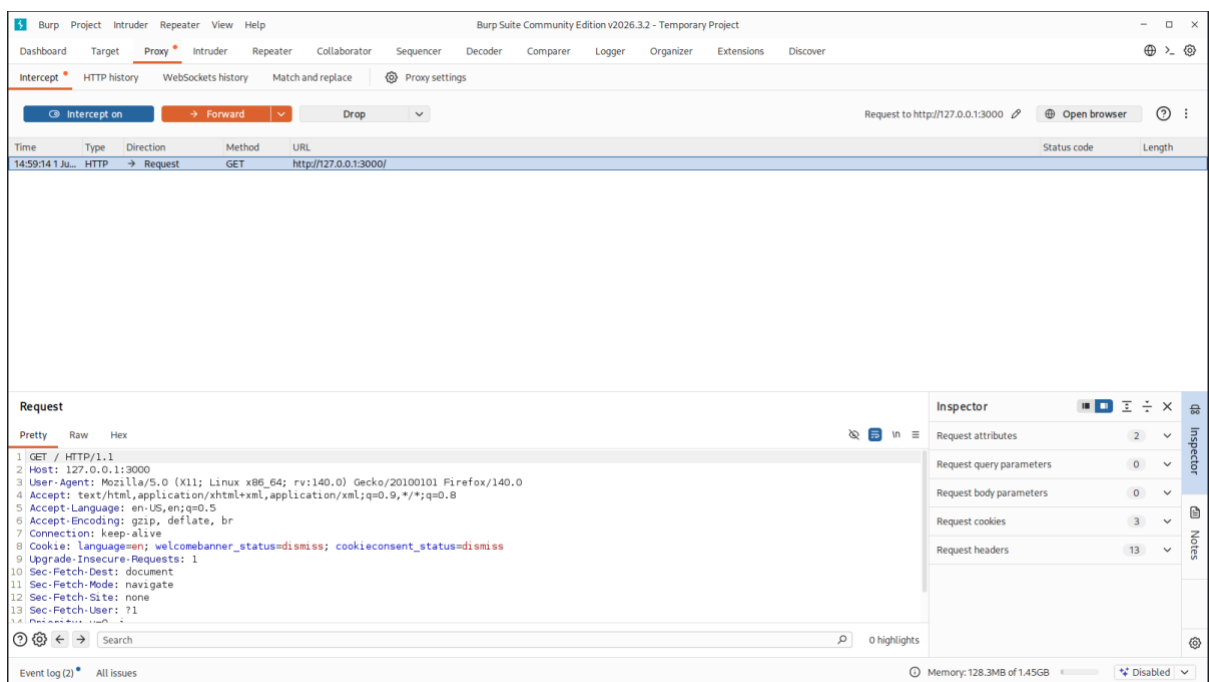


Figure 7: Intercepting HTTP Requests

Traffic Analysis and Endpoint Discovery

After successful proxy configuration, application traffic was analyzed through Burp Suite HTTP History.

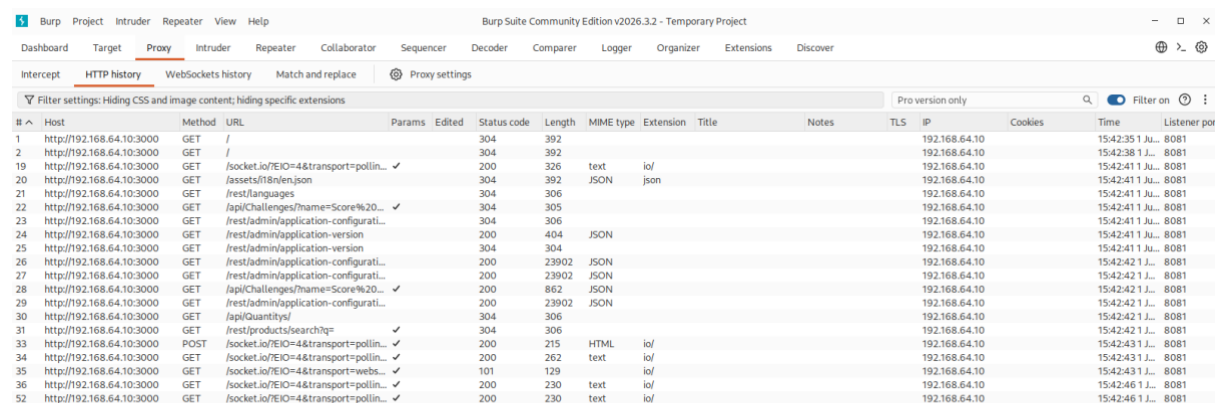
Important Endpoints Identified

```
"/api/Products"  
"/api/Challenges"  
"/api/BasketItems"  
"/api/Quantitys"  
"/rest/admin/application-configuration"
```

The discovered endpoints indicated the presence of:

- Product Management Functions
- User Authentication Components
- Basket Operations
- Administrative Configuration Features
- Application Challenge Tracking

These endpoints will be used during future vulnerability testing phases.



The screenshot shows the Burp Suite HTTP History tab with a list of intercepted requests. The table columns include #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Notes, TLS, IP, Cookies, Time, and Listener port. The requests are filtered to show only the 'Pro version only'.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port
1	http://192.168.64.10:3000	GET	/			304	392						192.168.64.10		15:42:35 1 Jul 8081	
2	http://192.168.64.10:3000	GET	/			304	392						192.168.64.10		15:42:38 1 Jul 8081	
19	http://192.168.64.10:3000	GET	/socket.io/?EIO=4&transport=pollin...		✓	200	326	text	io/				192.168.64.10		15:42:41 1 Jul 8081	
20	http://192.168.64.10:3000	GET	/assets/18/en.json			304	392	JSON	json				192.168.64.10		15:42:41 1 Jul 8081	
21	http://192.168.64.10:3000	GET	/rest/languages			304	306						192.168.64.10		15:42:41 1 Jul 8081	
22	http://192.168.64.10:3000	GET	/api/Challenges/?name=Score%20...		✓	304	305						192.168.64.10		15:42:41 1 Jul 8081	
23	http://192.168.64.10:3000	GET	/rest/admin/application-configuration...			304	306						192.168.64.10		15:42:41 1 Jul 8081	
24	http://192.168.64.10:3000	GET	/rest/admin/application-version			200	404	JSON					192.168.64.10		15:42:41 1 Jul 8081	
25	http://192.168.64.10:3000	GET	/rest/admin/application-version			304	304						192.168.64.10		15:42:41 1 Jul 8081	
26	http://192.168.64.10:3000	GET	/rest/admin/application-configuration...			200	23902	JSON					192.168.64.10		15:42:42 1 Jul 8081	
27	http://192.168.64.10:3000	GET	/rest/admin/application-configuration...			200	23902	JSON					192.168.64.10		15:42:42 1 Jul 8081	
28	http://192.168.64.10:3000	GET	/api/Challenges/?name=Score%20...		✓	200	862	JSON					192.168.64.10		15:42:42 1 Jul 8081	
29	http://192.168.64.10:3000	GET	/rest/admin/application-configuration...			200	23902	JSON					192.168.64.10		15:42:42 1 Jul 8081	
30	http://192.168.64.10:3000	GET	/api/Quantitys/			304	306						192.168.64.10		15:42:42 1 Jul 8081	
31	http://192.168.64.10:3000	GET	/rest/products/search?q=		✓	304	306						192.168.64.10		15:42:42 1 Jul 8081	
33	http://192.168.64.10:3000	POST	/socket.io/?EIO=4&transport=pollin...		✓	200	215	HTML	io/				192.168.64.10		15:42:43 1 Jul 8081	
34	http://192.168.64.10:3000	GET	/socket.io/?EIO=4&transport=pollin...		✓	200	262	text	io/				192.168.64.10		15:42:43 1 Jul 8081	
35	http://192.168.64.10:3000	GET	/socket.io/?EIO=4&transport=webs...		✓	101	129						192.168.64.10		15:42:43 1 Jul 8081	
36	http://192.168.64.10:3000	GET	/socket.io/?EIO=4&transport=pollin...		✓	200	230	text	io/				192.168.64.10		15:42:46 1 Jul 8081	
52	http://192.168.64.10:3000	GET	/socket.io/?EIO=4&transport=pollin...		✓	200	230	text	io/				192.168.64.10		15:42:46 1 Jul 8081	

Figure 8: Burp Suite HTTP History

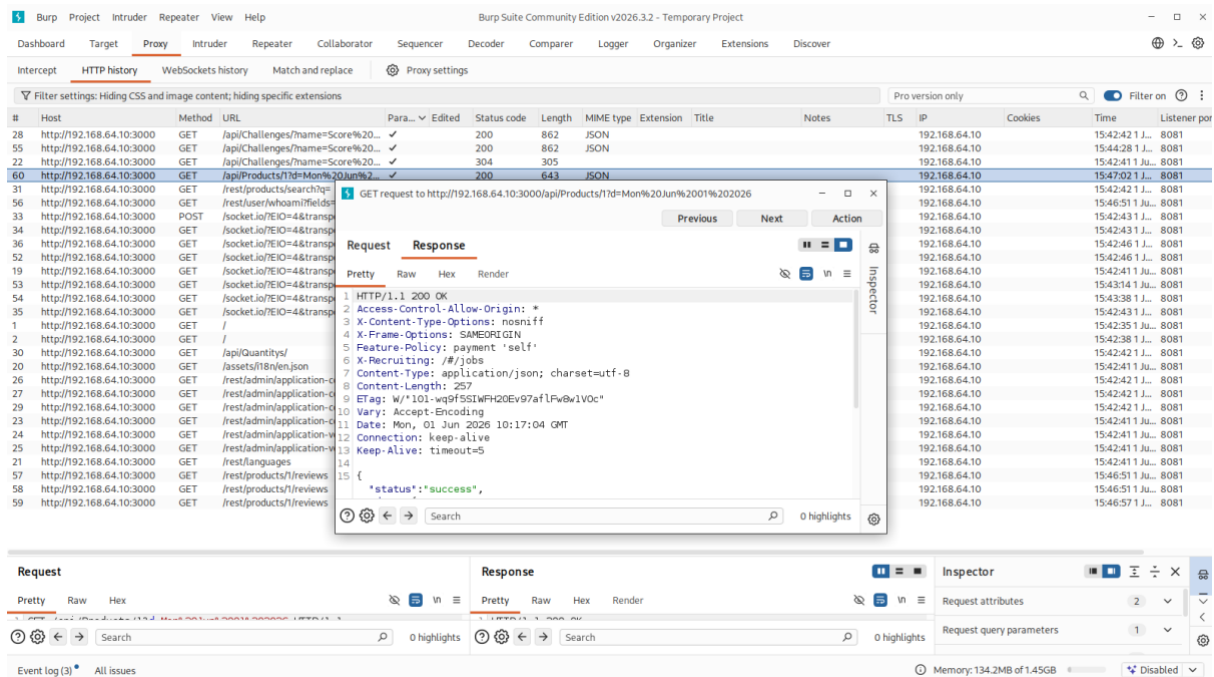


Figure 9: Captured Juice Shop API Requests

The Issues Encountered and Resolutions I Have Done In This Project

Issue 1: The Juice Shop was not Accessible due the faulty network setting issue from docker

Cause: Docker container was not running correctly.

Solution: Verified containers using Docker commands and restarted the application.

Result: Successfully accessed Juice Shop.

Issue 2: The Burp Suite was not Capturing Traffic

Cause: Firefox proxy settings were not configured correctly.

Solution: Configured Firefox to use Burp Proxy (127.0.0.1:8080).

Result: Burp Suite successfully captured HTTP requests and responses.

Issue 3: The empty site map in Burp Suite

Cause: Community Edition limitations.

Solution: Used HTTP History for traffic analysis.

Result: Successfully identified application endpoints.

During Week 1, the following observations were made:

- OWASP Juice Shop was successfully deployed using Docker.
- Apache and MariaDB services were operational.
- Nmap successfully identified active services.
- Burp Suite intercepted and analyzed web traffic successfully.
- Multiple API endpoints were discovered.
- The testing environment was successfully prepared for vulnerability assessment activities.

Outcome

The Week 1 objectives were successfully completed.

Achievements:

- OWASP Top 10 vulnerabilities studied.
- Vulnerable web applications deployed.
- Penetration testing environment configured.
- Reconnaissance completed using Nmap.
- Burp Suite configured successfully.
- Application endpoints identified and documented.
- Environment prepared for OWASP Top 10 vulnerability testing.

References link

1. OWASP Top 10 Project – <https://owasp.org/www-project-top-ten/>
2. OWASP Juice Shop – <https://owasp.org/www-project-juice-shop/>
3. PortSwigger Burp Suite Documentation – <https://portswigger.net/burp/documentation>
4. Nmap Official Documentation – <https://nmap.org/book/man.html>
5. Docker Documentation – <https://docs.docker.com/>